PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS
(PUC-Campinas)

Projeto Integrador (PI): Desenvolvimento de Sistemas Orientados a Objetos — Prática
(ID: 218040P)

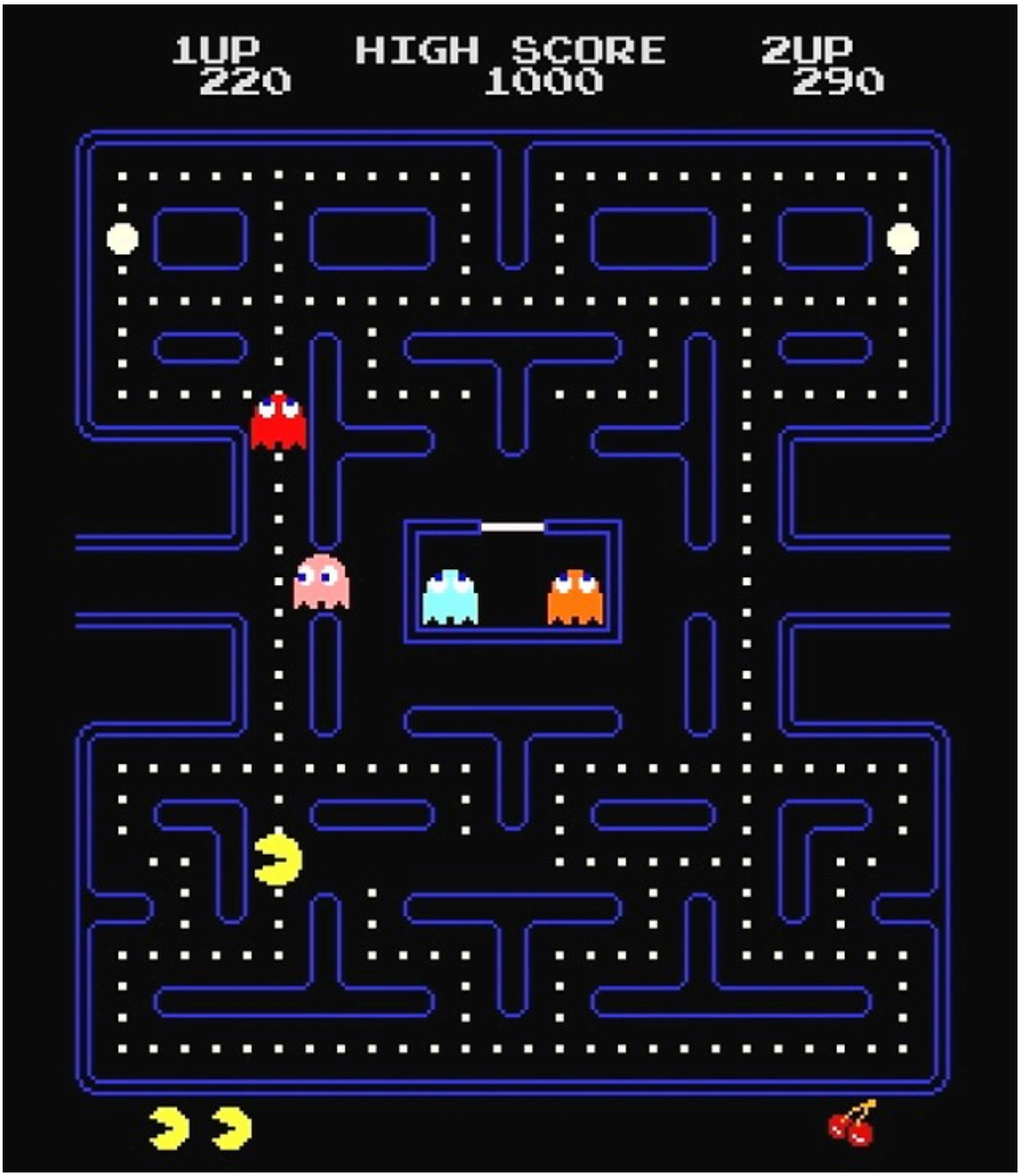
Professor responsável: Leonardo Rezende Costa (leonardo.costa@puc-campinas.edu.br)

Integrantes do grupo:

Nome	E-mail	RA
João Paulo Nunes Andrade	joao.pna@puccampinas.edu.br	25002703
Gabriel de Oliveira Baptista	gabriel.ob1@puccampinas.edu.br	25014604
Gustavo Antoniazzi Gouveia Santos	gustavo.ags@puccampinas.edu.br	25007671
Enzo Marchi Romera	enzo.mr@puccampinas.edu.br	25009691
GitHub	https://github.com/joaopaulonu/pacman-atari-2600	

Campinas, 2026

Proposta de Projeto: Pac-Man OOP



Disponível em: https://cemeterygames.com/wp-content/uploads/2010/05/pac_man.jpg

1. Nome do Jogo

Pac-Man OOP

O nome diz o que queremos fazer: uma releitura do Pac-Man da versão de Atari 2600, publicada em 1982 e programada por Tod Frye [1], em que o foco está na estrutura do código e não só no resultado visual. A versão original do console teve de se virar com um cartucho de 4 KB e apenas 128 bytes de RAM [1][3], e isso gerou soluções peculiares, como os fantasmas que cintilam porque só um é desenhado por quadro. Nosso projeto parte do mesmo jogo, mas sem essas amarras de hardware, o que nos deixa investir o esforço na modelagem de classes.

2. Gênero

Labirinto / Arcade 2D, para um jogador, com visão de cima e movimentação em grade.

3. Resumo da Ideia do Jogo

O jogador controla o Pac-Man, um personagem circular e amarelo que começa cada partida no centro de um labirinto. O objetivo é percorrer os corredores e comer todos os biscoitos espalhados pelo mapa. Quando o último biscoito some, a fase termina e o jogo avança para a próxima.

O problema são os quatro fantasmas que patrulham o labirinto. O contato com qualquer um deles custa uma vida, e o Pac-Man reaparece no ponto inicial. Quando as vidas acabam, o jogo termina. Para equilibrar a disputa, há quatro pílulas de energia, uma em cada canto do labirinto. Ao comer uma delas, o Pac-Man fica invulnerável por alguns segundos e a dinâmica se inverte: os fantasmas passam a fugir e podem ser comidos, o que rende pontos extras [1][3]. Depois que o efeito acaba, tudo volta ao normal. Itens especiais que aparecem de vez em quando no mapa também dão pontuação adicional.

Na versão de Atari, os itens do mapa tinham nomes e aparência diferentes dos do fliperama (biscoitos retangulares marrons, por exemplo), embora cumprissem as mesmas funções [1]. Vamos manter esse espírito visual simples. Quanto ao efeito de cintilação dos fantasmas, ele nasceu de uma limitação técnica e não é prioridade para nós. Se sobrar tempo no cronograma, pode entrar como opção de "modo retrô".

4. Mapeamento Inicial de Entidades

Dividimos o sistema em dois grupos. Os **elementos interativos** são as coisas que existem dentro do labirinto e reagem entre si. Os **gerenciadores** não aparecem na tela, mas cuidam das regras e do fluxo da partida. O mapeamento abaixo é inicial e deve mudar conforme a implementação avançar.

4.1 Elementos interativos

Todos os elementos do mapa herdam de uma classe base abstrata, **`EntidadeJogo`**. Ela guarda o que qualquer objeto do jogo precisa ter: posição na grade, sprite, estado de ativo/inativo e os métodos `atualizar()`, `desenhar()` e `colideCom(outra)`. Cada subclasse sobrescreve o que lhe cabe, e o laço principal do jogo trata todas as entidades pela mesma interface, sem precisar saber o tipo concreto de cada uma. É nisso que o polimorfismo vai aparecer com mais força.

Jogador (`PacMan`). Herda de `EntidadeJogo` e, por se mover, passa pela subclasse intermediária `EntidadeMovel`, que concentra velocidade, direção e checagem de paredes. O `PacMan` acrescenta a leitura do teclado (ou joystick), o número de vidas e o estado de invulnerabilidade. Ao colidir com um fantasma, o resultado depende desse estado: se estiver invulnerável, come o fantasma; se não, perde uma vida.

Fantasmas (`Fantasma`). Também herdam de `EntidadeMovel`. Cada um dos quatro tem uma estratégia própria de patrulha, e queremos modelar isso com uma classe abstrata `Fantasma` e quatro subclasses, cada uma sobrescrevendo o método que escolhe a próxima direção (`escolherDirecao()`). O comportamento geral compartilhado fica na classe-mãe. Cada fantasma tem pelo menos três estados: *patrulha*, *assustado* (durante o efeito da pílula) e *comido* (retornando ao ponto de origem). A troca de estado será feita pelo padrão State, para evitar uma cadeia de `if` espalhada pelo código.

Biscoitos e pílulas de energia (`Coletavel`). São entidades fixas, então herdam diretamente de `EntidadeJogo`, por meio da classe abstrata `Coletavel`, que define o valor em pontos e o método `aoSerColetado(jogador)`. Duas subclasses concretas especializam o comportamento:

- `Biscoito`: apenas soma pontos e se desativa.
- `PilulaEnergia`: soma pontos, ativa a invulnerabilidade do jogador e avisa os fantasmas para mudarem de estado.

Se quisermos incluir os itens bônus ocasionais, eles entram como uma terceira subclasse de `Coletavel`, sem nenhuma mudança nas classes existentes.

4.2 Gerenciadores

`GameManager` (Gerenciador de Jogo). É o ponto de entrada da lógica. Controla o estado geral da partida (menu, jogando, pausado, fim de jogo), executa o laço principal, dispara a atualização e o desenho das entidades e decide quando a fase acabou ou quando o jogador perdeu. Também coordena os outros gerenciadores.

`ScoreManager` (Gerenciador de Placar). Mantém a pontuação atual, o recorde e o contador de vidas exibido na interface. Recebe eventos como "biscoito coletado" ou "fantasma comido" e atualiza o placar de acordo. Deve também aplicar a regra de pontuação crescente quando vários fantasmas são comidos numa mesma pílula, se decidirmos adotá-la.

`MazeManager` (Gerenciador de Labirinto/Fase). Carrega o mapa de cada fase a partir de uma matriz (a ideia é ler de um arquivo de texto, para facilitar a criação de novos níveis), instancia paredes, biscoitos, pílulas e pontos de origem e responde a consultas de colisão do tipo "esta célula é uma parede?". Também sabe quantos biscoitos restam, e é daí que sai a condição de vitória da fase.

`SoundManager` (Gerenciador de Áudio). Centraliza a execução de efeitos sonoros e trilhas (início de fase, coleta de biscoito, pílula ativada, fantasma comido, morte do jogador). As demais classes não tocam som diretamente; elas notificam o gerenciador, o que mantém o acoplamento baixo e permite silenciar tudo num único lugar.

4.3 Como as peças se conectam

Os gerenciadores se comunicam por eventos simples (por exemplo, o `PacMan` avisa que coletou algo e o `ScoreManager` e o `SoundManager` reagem), e não por referências diretas entre todos. A ideia é que adicionar um novo tipo de fantasma, um novo item ou uma nova fase exija mexer no mínimo possível de código já pronto. Essa é a principal métrica que vamos usar para avaliar se a nossa modelagem funcionou.

Referências Bibliográficas

[1] Wikipédia. **Pac-Man (Atari 2600)**. Disponível em: <https://pt.wikipedia.org/wiki/Pac-Man_(Atari_2600)>.

[2] Pac-Man Official Website. **History**. Disponível em: <https://pacman.com/en/history/>.

[3] Pac-Man Fandom Wiki. **Pac-Man (Atari 2600)**. Disponível em: <https://pacman.fandom.com/wiki/Pac-Man_(Atari_2600)>.